

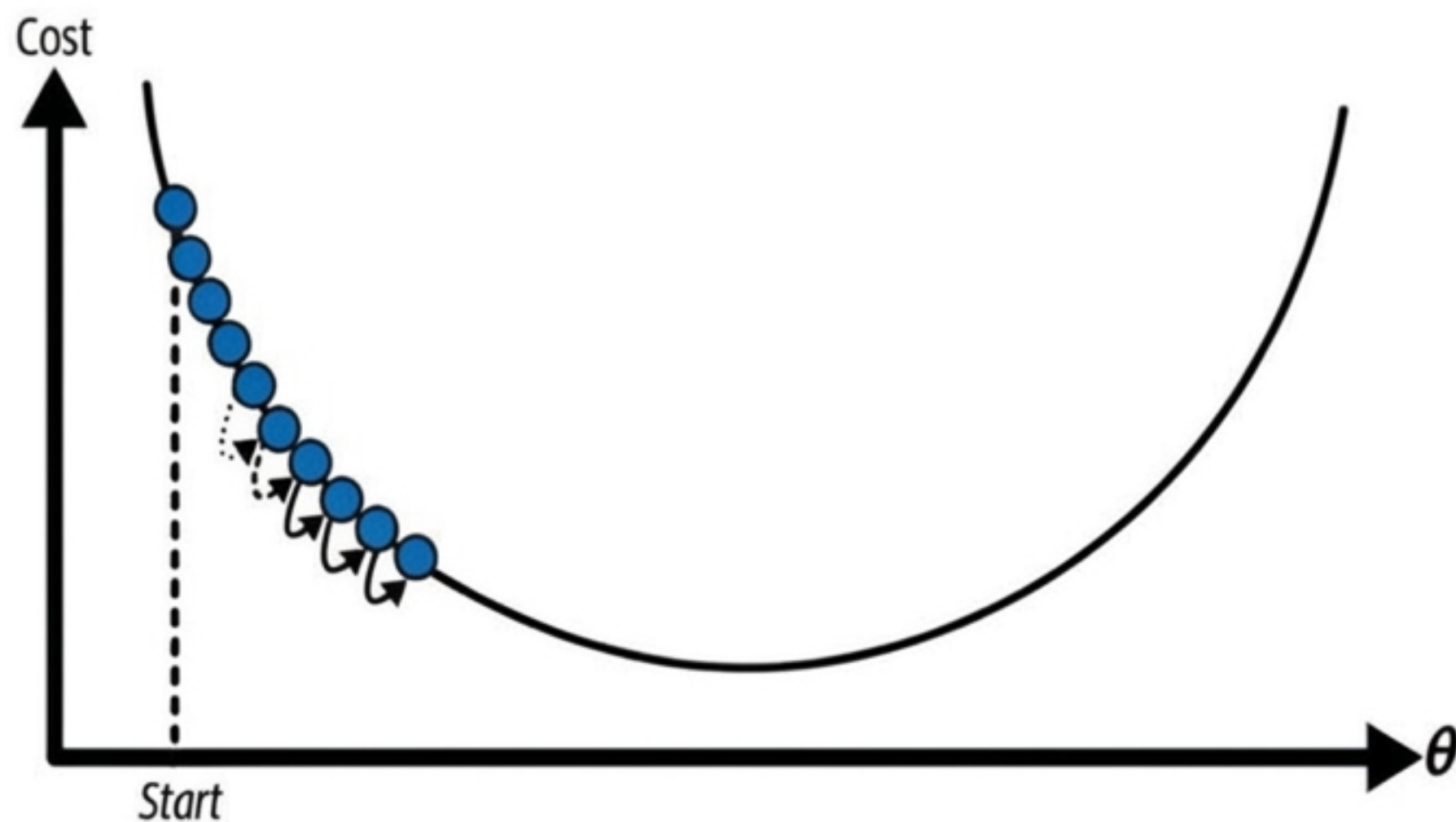
TÀI LIỆU BÀI GIẢNG BẬC ĐẠI HỌC

MÁY HỌC - CHƯƠNG 4 (PHẦN 2)

Tối ưu hóa Gradient Descent & Hồi quy Đa thức



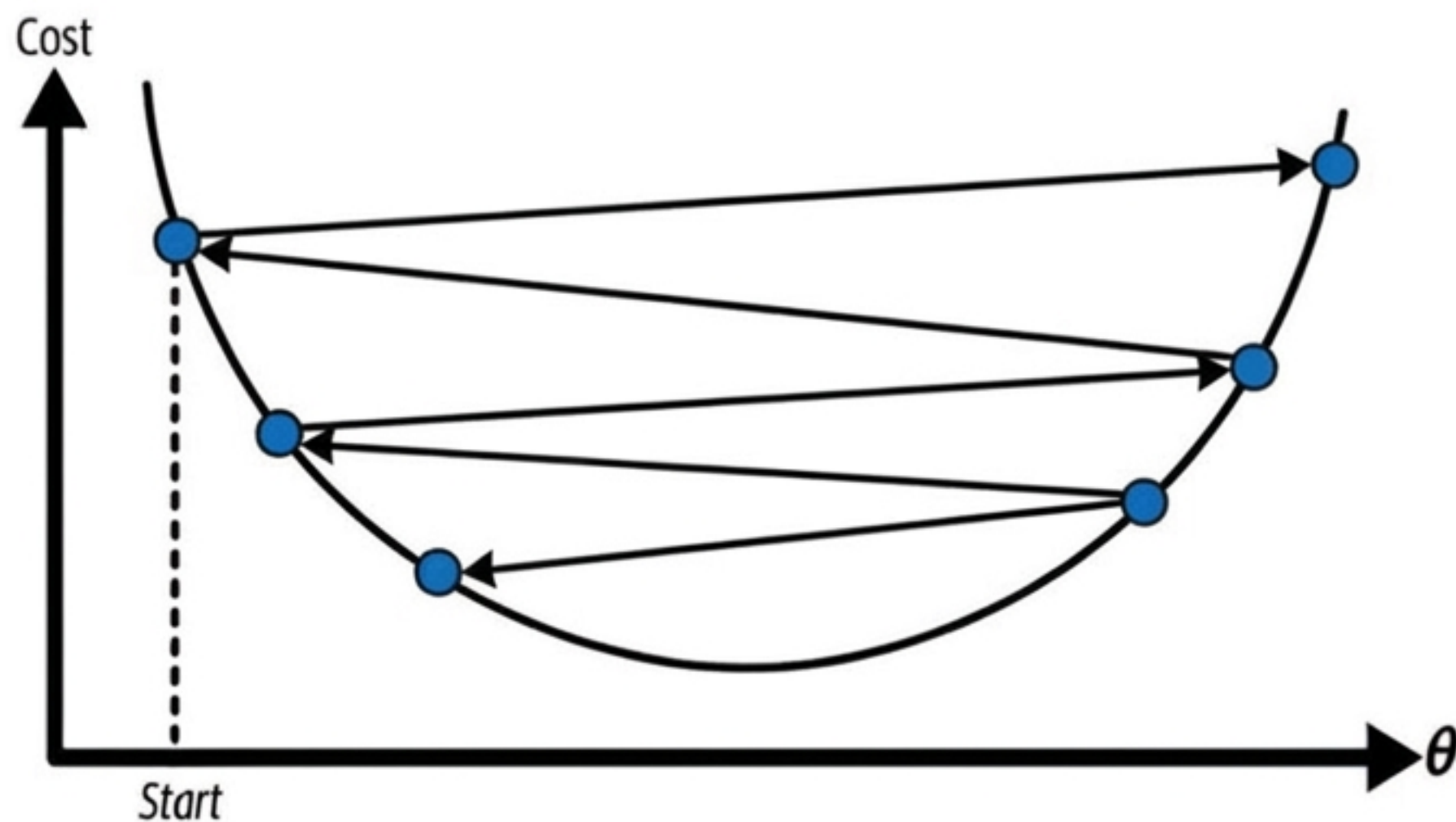
Minh họa: Tốc độ học quá nhỏ



Hình 4-4. Tốc độ học quá nhỏ

- **Đặc điểm:** Tốc độ học (Learning Rate) được thiết lập ở mức quá thấp.
- **Hậu quả:**
 - Mô hình vẫn sẽ đạt được cực tiểu.
 - Tuy nhiên, đường đi rất chậm, chậm, mất quá nhiều thời gian và số bước lặp để hội tụ.

Minh họa: Tốc độ học quá lớn

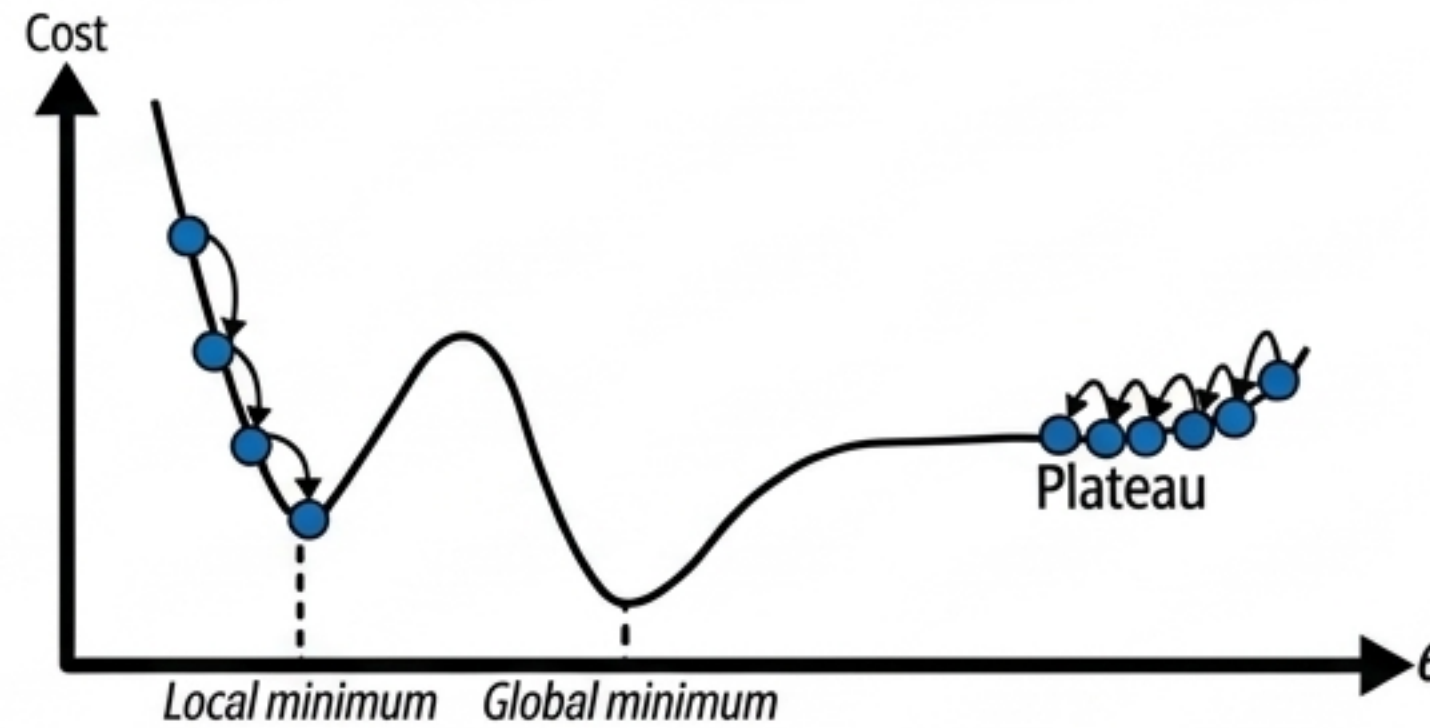


Hình 4-5. Tốc độ học quá lớn

- **Đặc điểm:** Tốc độ học được (Learning Rate) được thiết lập ở mức quá cao.
- **Hậu quả:**
 - Thuật toán liên tục vượt qua điểm cực tiểu.
 - Thay vì hội tụ, thuật toán nhảy loạn xạ và đang phân kỳ (diverging).
 - Hàm chi phí ngày càng tăng lên.

Các cạm bẫy của Gradient Descent

Không phải tất cả các hàm chi phí đều giống như những cái bát hình parabol hoàn hảo.



Hình 4-6. Cạm bẫy cực tiểu cục bộ

Cực tiểu cục bộ (Local Minimum):

Nếu bắt đầu từ điểm ngẫu nhiên, thuật toán có thể hội tụ về điểm thấp cục bộ chứ không phải cực tiểu toàn cục (Global Minimum).

Cao nguyên (Plateau): Hàm chi phí đi ngang, khiến thuật toán mất rất nhiều thời gian đi qua, dễ bị dừng lại sớm.

Hàm lồi (Convex function): Rất may, hàm chi phí MSE của hồi quy tuyến tính là hàm lồi (luôn có một cực tiểu toàn cục duy nhất).

Tầm quan trọng của việc điều chỉnh đặc trưng (Scaling)

Khi sử dụng Gradient Descent, bạn phải đảm bảo tất cả các đặc trưng có thang đo (scale) tương tự nhau (VD: dùng StandardScaler).



CÓ SCALING

- Các đặc trưng được chia tỷ lệ bằng nhau.
- Hàm chi phí có dạng cái bát tròn.
- Đường đi đến cực tiểu nhanh gọn và thẳng.



KHÔNG CÓ SCALING

- Đặc trưng 1 nhỏ hơn Đặc trưng 2.
- Hàm chi phí có dạng bát kéo dài.
- Đường đi ngoằn ngoèo, tốn nhiều thời gian hội tụ.

Minh họa khái niệm cho: Hình 4-7. Gradient Descent với và không có Scaling

Hạ Gradient theo lô (Batch Gradient Descent)



Tính toán đạo hàm riêng của hàm chi phí đối với mỗi tham số θ_j bằng cách sử dụng toàn bộ tập dữ liệu huấn luyện tại mỗi bước lặp.

VECTOR GRADIENT CỦA HÀM CHI PHÍ

$$\nabla_{\theta\text{MSE}}(\boldsymbol{\theta}) = \frac{2}{m} \mathbf{X}^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y}) \quad (5)$$

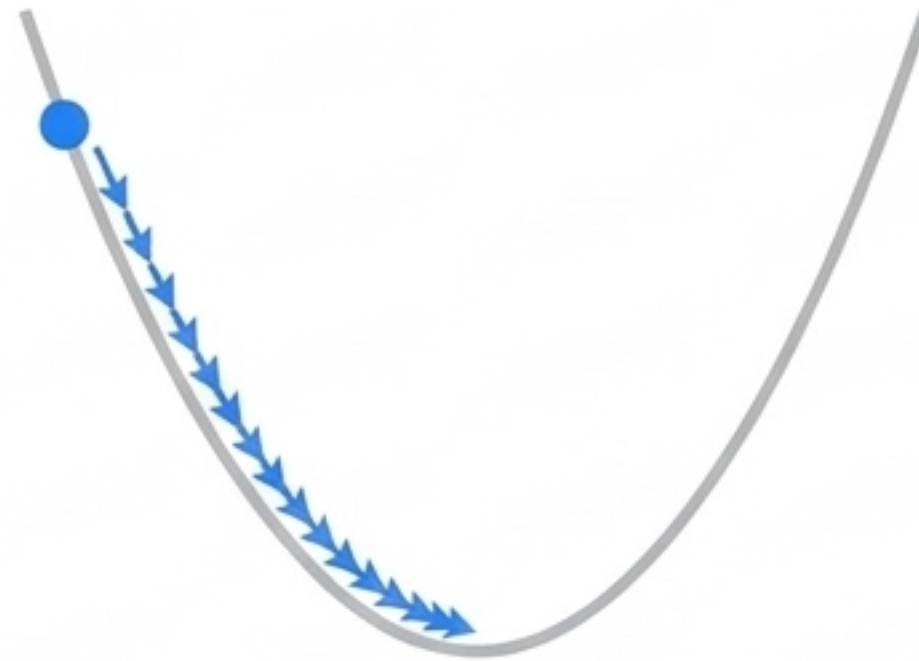
BƯỚC NHẢY GRADIENT DESCENT

$$\boldsymbol{\theta}(\text{bước tiếp theo}) = \boldsymbol{\theta} - \eta \nabla_{\theta\text{MSE}}(\boldsymbol{\theta}) \quad (6)$$

Mặc dù **chậm trên tập dữ liệu rất lớn** (phải dùng toàn bộ dữ liệu), nhưng **chia tỷ lệ rất tốt với số lượng đặc trưng lớn** (tốt hơn **Phương trình chuẩn tắc**).

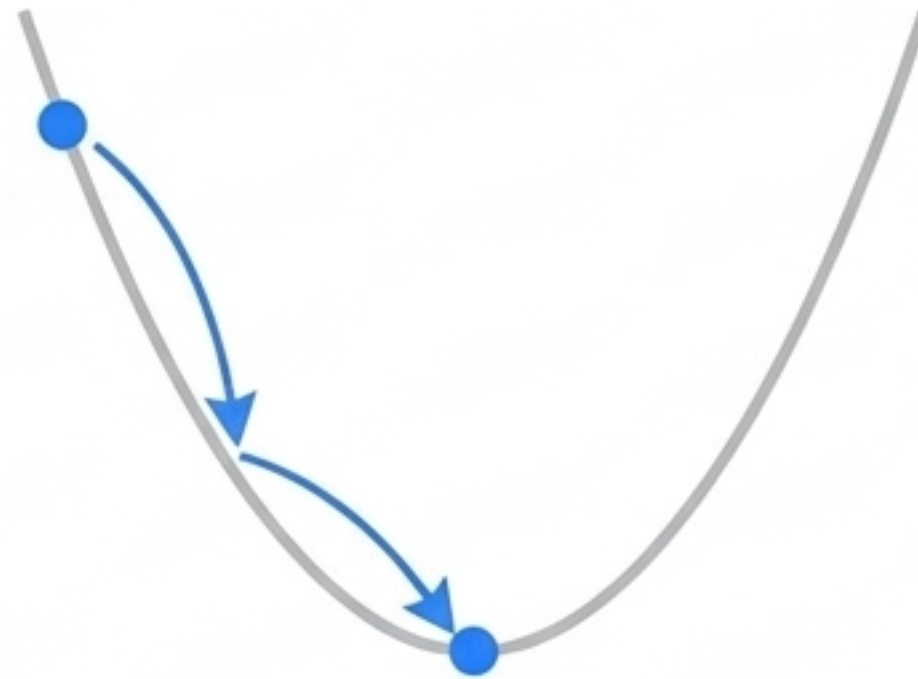
Minh họa: Batch GD với các tốc độ học khác nhau

Xét 10 bước đầu tiên của Gradient Descent theo lô:



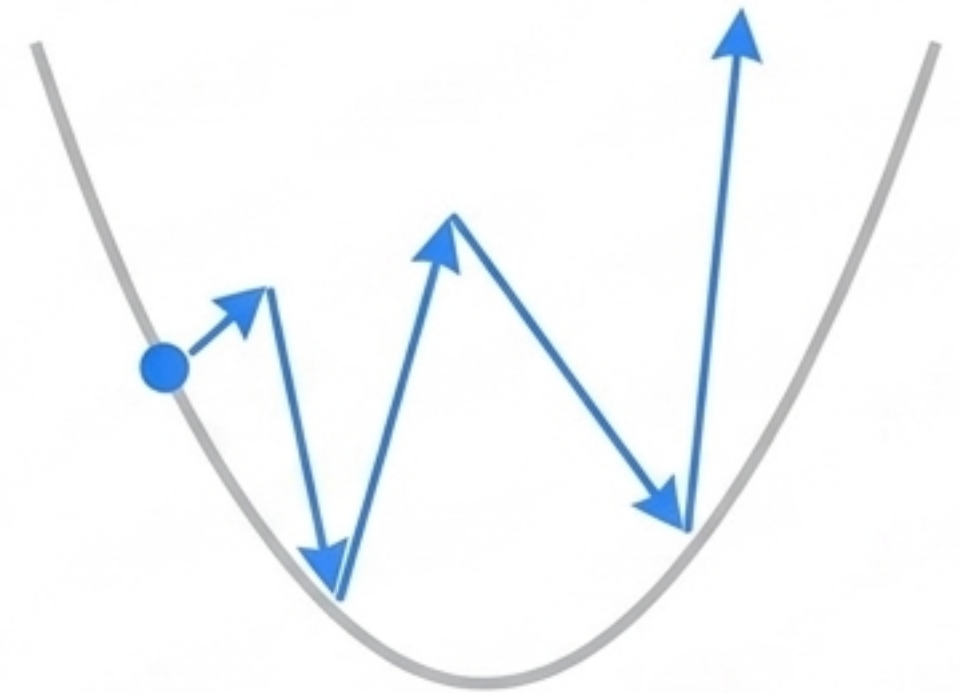
Trái ($\eta = 0.02$):

- Tốc độ học quá thấp.
- Sẽ đạt cực tiểu nhưng tiến trình rất chậm.



Giữa ($\eta = 0.1$):

- Tốc độ học rất tốt.
- Thuật toán hội tụ nhanh chóng chỉ trong vài bước.



Phải ($\eta = 0.5$):

- Tốc độ học quá cao.
- Thuật toán nhảy loạn xạ và đang phân kỳ (diverging).

Trích xuất nội dung từ: Hình 4-8. Batch GD với tốc độ học khác nhau

Hạ Gradient ngẫu nhiên (Stochastic Gradient Descent - SGD)

Vấn đề: Vấn đề chính của **Batch GD** là sử dụng toàn bộ tập dữ liệu huấn luyện để tính gradient tại mỗi bước, làm nó rất chậm với tập dữ liệu lớn.

Giải pháp SGD:

- Chọn ra một trường hợp ngẫu nhiên trong tập huấn luyện tại mỗi bước.
- Chỉ tính gradient trên trường hợp duy nhất đó.

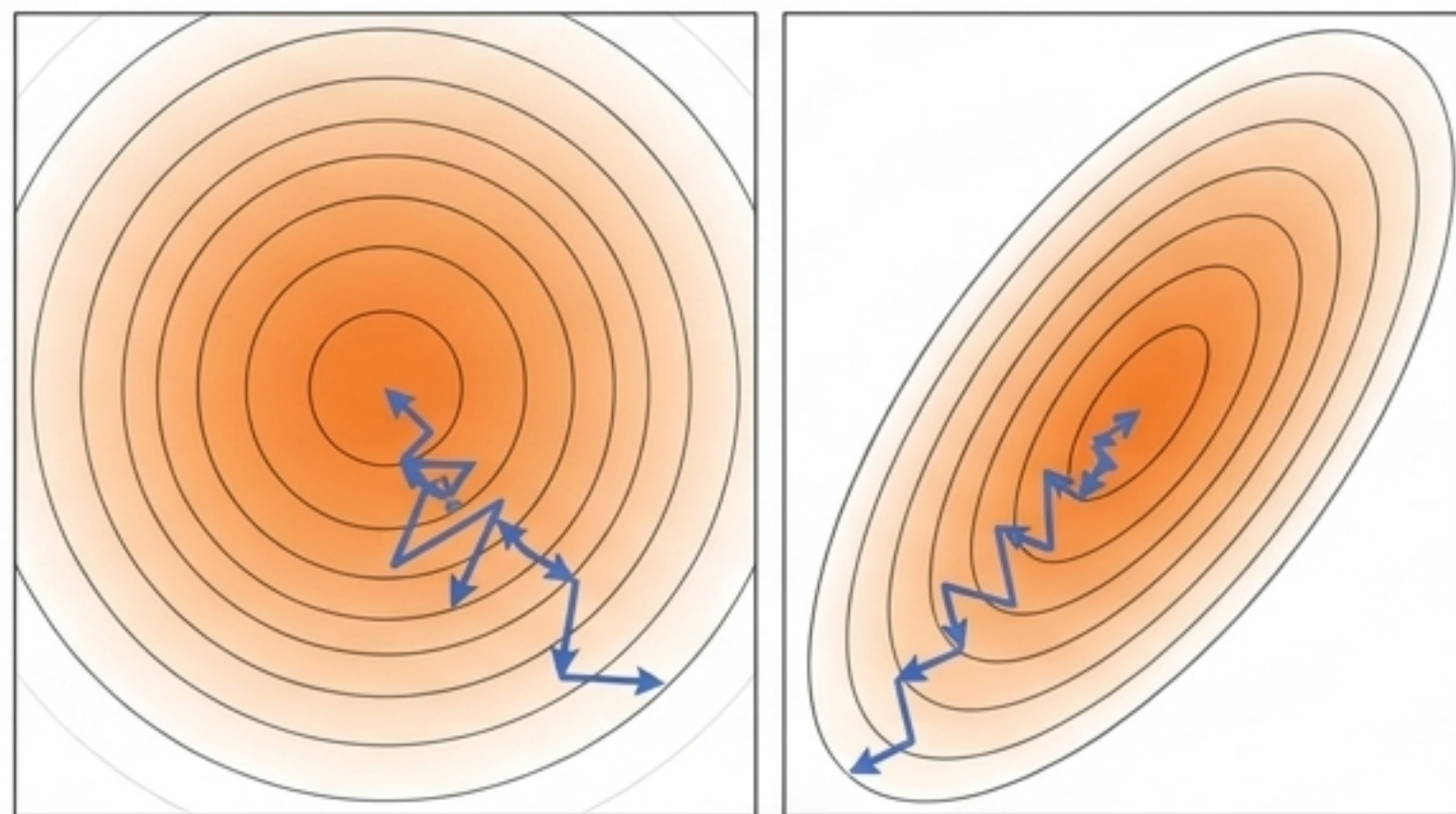
Ưu điểm vượt trội:

- Thuật toán trở nên cực kỳ nhanh.
- Khả năng thực hiện huấn luyện các mô hình rất lớn vì mỗi lần lặp chỉ cần 1 trường hợp trong bộ nhớ.
- Hỗ trợ học ngoài lõi (Out-of-core learning).



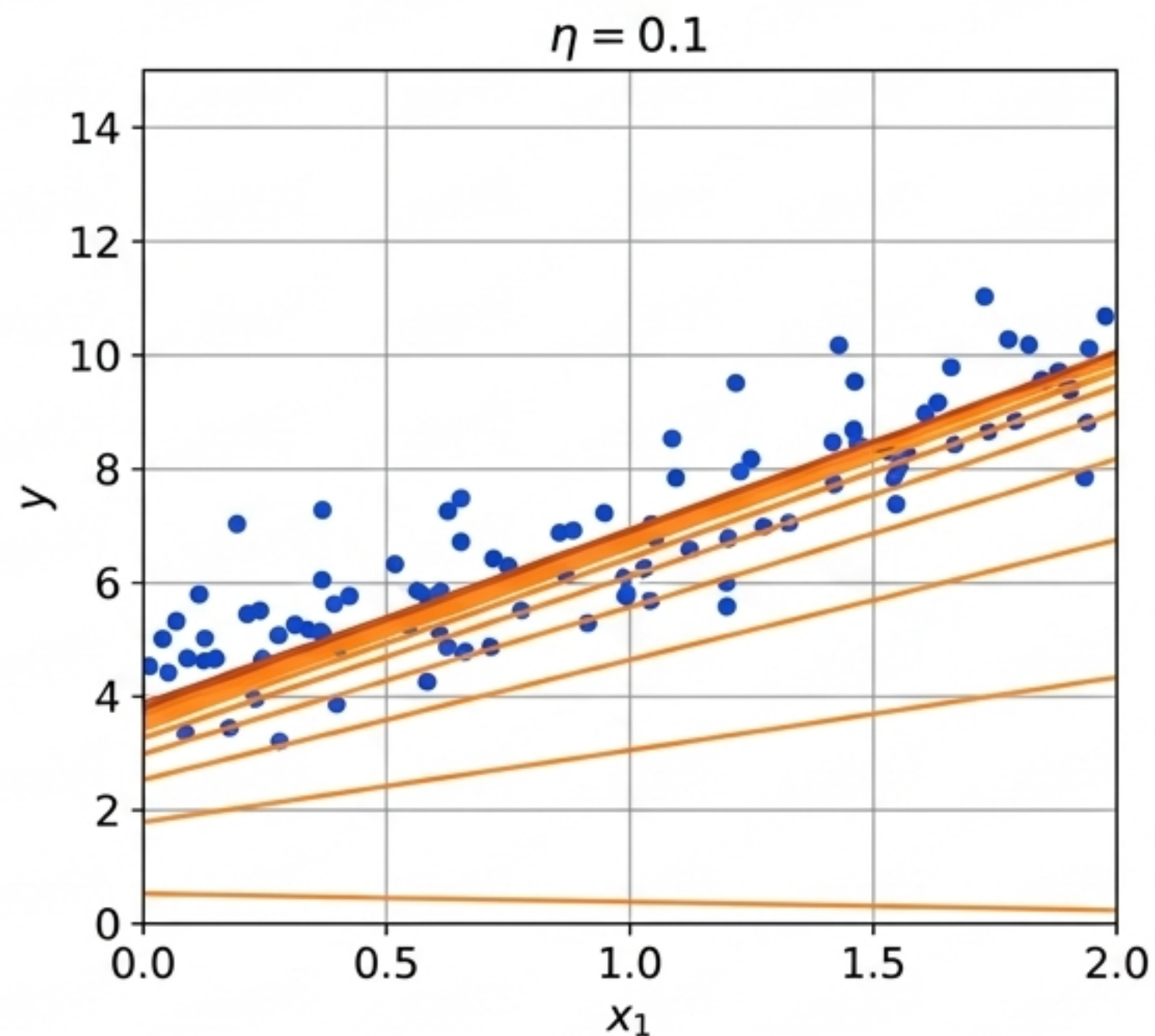
Đặc điểm đường đi của Stochastic Gradient Descent

- Do tính chất ngẫu nhiên, SGD thất thường hơn nhiều so với Batch GD.
- Hàm chi phí dao động lên xuống, giảm dần về trung bình thay vì giảm nhẹ nhàng.
- Ngay cả khi sát cực tiểu, tham số vẫn tiếp tục dao động.
- **Kết quả:** Giá trị cuối cùng là tốt, nhưng chưa phải tối ưu tuyệt đối.
- **Ưu điểm:** Sự ngẫu nhiên giúp thuật toán nhảy ra khỏi các cực tiểu cục bộ tốt hơn Batch GD.



Hình 4-9. Đặc điểm bất thường của SGD

Minh họa: 20 bước đầu tiên của SGD



Hình 4-10. 20 bước đầu tiên của SGD

Hình ảnh minh họa 20 bước đầu tiên của SGD. Các bước không đều đặn, nhảy liên tục so với Batch GD.

Giải pháp: Lịch học (Learning schedule)

Để SGD hội tụ sát cực tiểu hơn, áp dụng kỹ thuật giảm tốc độ học:

- **Lúc đầu:** Tốc độ học lớn (giúp tránh kẹt tại cực tiểu cục bộ).
- **Về sau:** Giảm dần tốc độ học khi tiến về gần đích để ổn định tại đáy thung lũng.

Hạ Gradient theo Mini-Batch (Mini-Batch GD)

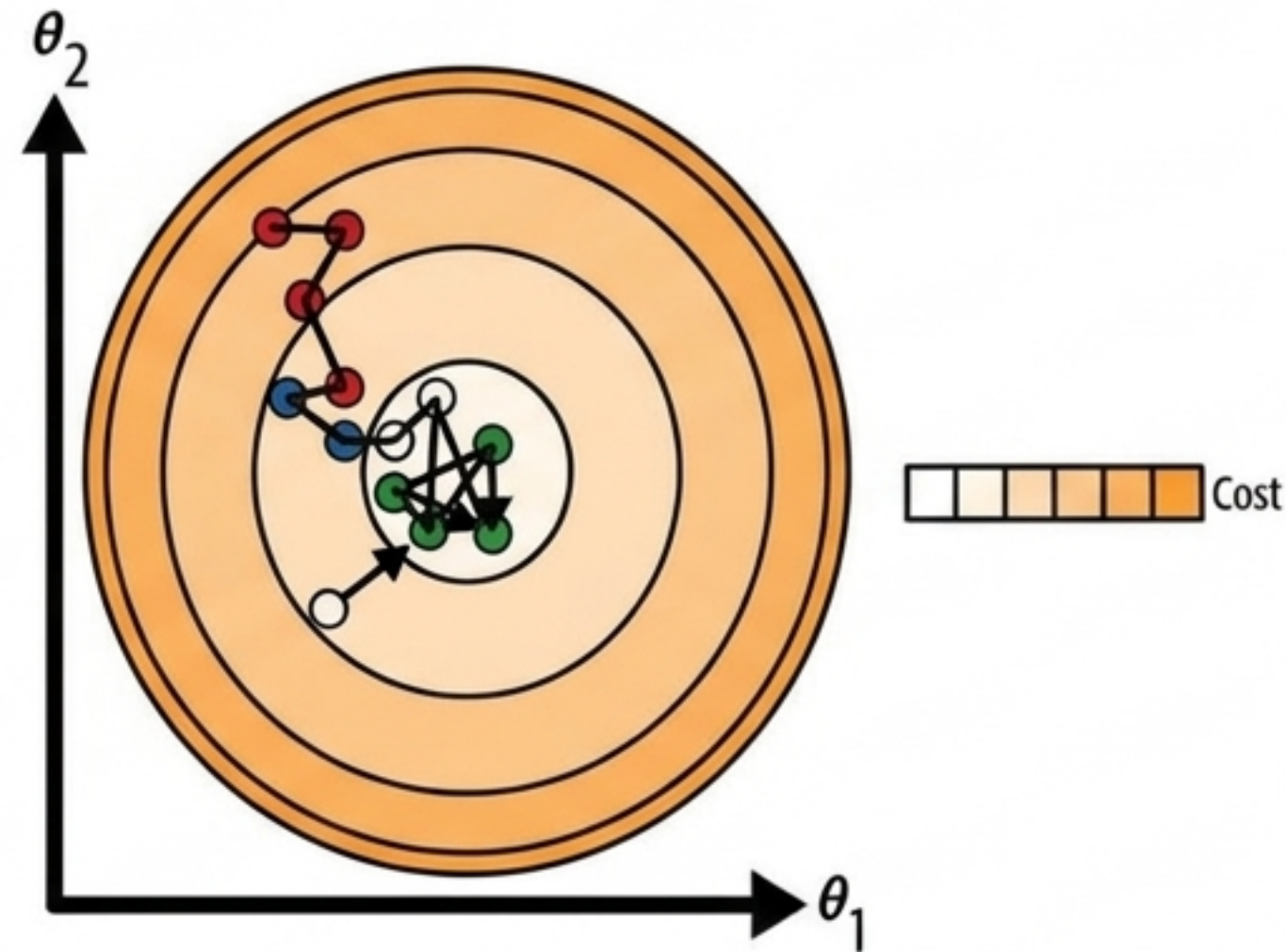
Khái niệm: Tại mỗi bước, thay vì tính toán dựa trên toàn bộ dữ liệu (Batch) hay 1 trường hợp (SGD), thuật toán tính gradient trên các tập con nhỏ ngẫu nhiên gọi là mini-batch.

Đặc điểm & Ưu điểm:

- **Tối ưu phần cứng:** Tăng hiệu suất từ việc tối ưu hóa các phép toán ma trận, đặc biệt khi sử dụng GPUs.
- **Mức độ dao động:** Sự tiến bộ của thuật toán ít thất thường hơn SGD (với mini-batch khá lớn).
- **Hội tụ:** Tiến gần cực tiểu hơn SGD một chút.
- **Nhược điểm:** Có thể khó thoát khỏi cực tiểu cục bộ hơn so với SGD.



So sánh đường đi của 3 thuật toán Gradient Descent



Hình 4-11. Đường đi của 3 thuật toán GD

Batch GD (Đỏ): Điềm đến thẳng cực tiểu và dừng lại, nhưng tốn nhiều thời gian mỗi bước.

SGD (Trắng): Dao động mạnh xung quanh đích.

Mini-Batch GD (Xanh): Ít dao động hơn SGD và tiến gần đích hơn, nhưng vẫn không dừng lại chính xác tại một điểm trừ khi có lịch học (learning schedule) tốt.

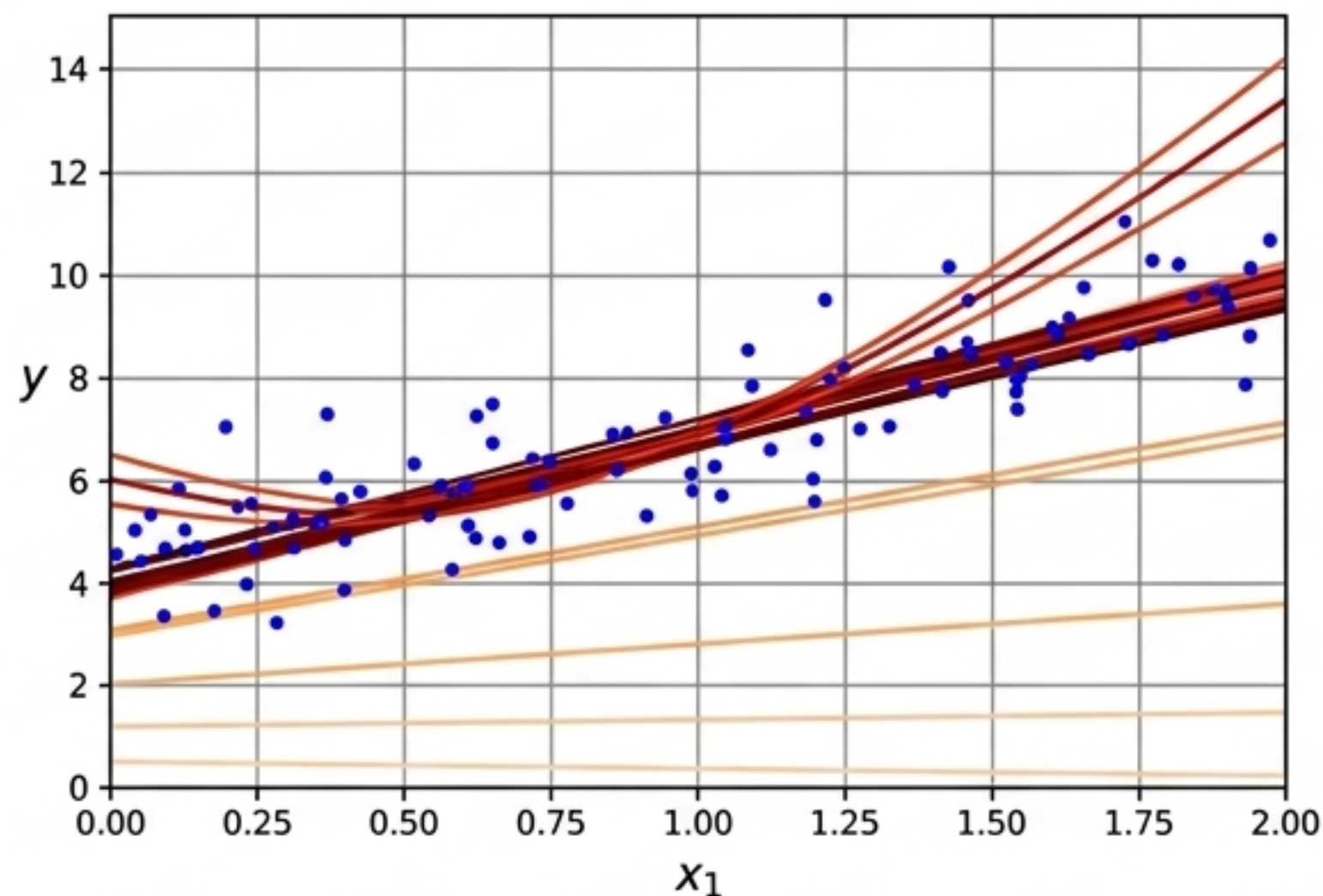
Bảng so sánh tổng hợp các thuật toán

Thuật toán	Lớn m (trường hợp)	Ngoài lỗi	Lớn n (đặc trưng)	Hyperparams	Scaling	Scikit-Learn
Phương trình chuẩn tắc	Nhanh	Không	Chậm	0	Không	N/A
SVD	Nhanh	Không	Chậm	0	Không	LinearRegression
Batch GD	Chậm	Không	Nhanh	2	Có	N/A
Stochastic GD	Nhanh	Có	Nhanh	≥ 2	Có	SGDRegressor
Mini-batch GD	Nhanh	Có	Nhanh	≥ 2	Có	N/A

Lưu ý: Sau khi huấn luyện, tất cả các thuật toán này đều tạo ra mô hình rất giống nhau và đưa ra dự đoán theo cùng một cách.

3. Hồi quy đa thức, Đường cong học tập & Chính quy hóa

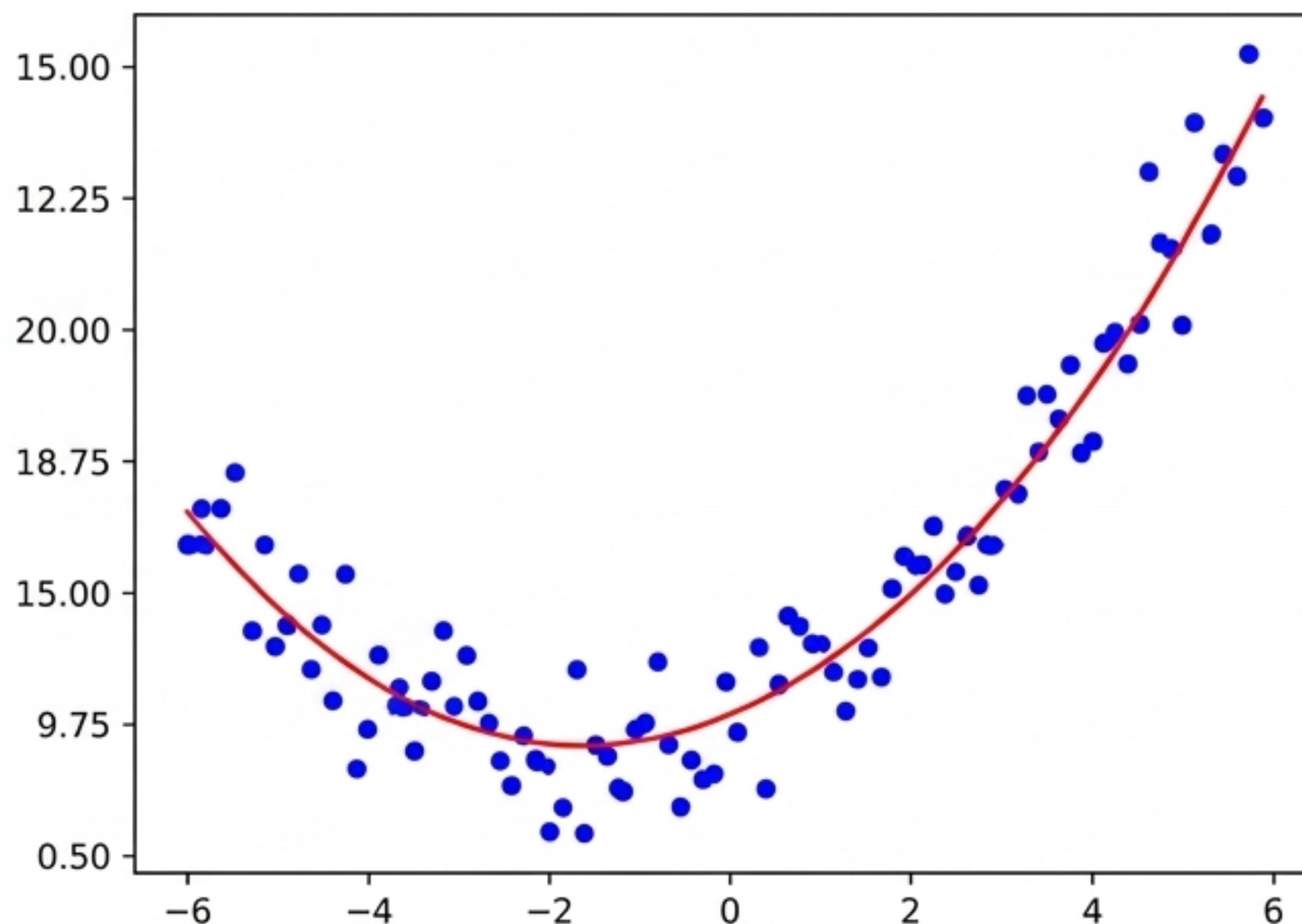
- **Vấn đề:** Điều gì xảy ra nếu dữ liệu phức tạp hơn một đường thẳng? (Underfitting với hồi quy tuyến tính).
- **Giải pháp - Hồi quy đa thức:** Thêm các lũy thừa của mỗi đặc trưng làm đặc trưng mới, sau đó huấn luyện mô hình tuyến tính trên tập đặc trưng mở rộng này.
- **Công cụ:** PolynomialFeatures trong Scikit-Learn tạo ra các biến bậc cao và kết hợp biến.
- **Ví dụ:** Tập dữ liệu phân bố theo phương trình bậc hai ($y = ax^2 + bx + c + \text{nhiều}$).



Hình 4-12. Tập dữ liệu phi tuyến tính ngẫu nhiên

Khớp Hồi quy đa thức trên dữ liệu phi tuyến

- Một đường thẳng tuyến tính bình thường không bao giờ khớp được dữ liệu này.
- Bằng cách dùng `PolynomialFeatures(degree=2)`, chúng ta thêm x^2 vào bộ đặc trưng.
- Sau đó dùng `LinearRegression` khớp dữ liệu mới.
- **Kết quả:** Thu được là một đường parabol bám rất sát vào phân bố dữ liệu ban đầu.



Hình 4-13. Dự đoán mô hình hồi quy đa thức